SESD-Assignment-6

G.Sanjansah.

2503A52L20.

1. Singleton – Banking Management System
Develop a banking system with a centralized BankManager responsible for managing common banking operations and system configuration. Implement BankManager using Singleton and demonstrate that only one instance is created.

2. Factory Method – Travel Booking System
Develop a travel booking system supporting Flight, Train, and Bus bookings. Use Factory Method to create the appropriate booking object based on the customer's selection.

Implement using: Python or Java.

1. Aim

To implement and demonstrate the Singleton and Factory Method Design Patterns using Python.

2. Singleton – Banking Management System

Objective: To develop a banking system where only one instance of the `BankManager` class is created.

Procedure:

1. Create a `BankManager` class using the Singleton Design Pattern.
2. Use the `__new__()` method to control object creation.
3. Implement deposit, withdrawal, and balance-checking operations.
4. Create two objects of the `BankManager` class.
5. Verify that both variables refer to the same instance using the `is` operator.
6. Execute the program and verify the output.

Result: Successfully implemented the Singleton Design Pattern and verified that only one instance of `BankManager` is created.

3. Factory Method – Travel Booking System

Objective: To develop a travel booking system supporting Flight, Train, and Bus bookings using a factory method.

Procedure:

1. Create an abstract `Booking` class.
2. Implement `FlightBooking`, `TrainBooking`, and `BusBooking` classes.
3. Create a `BookingFactory` class to generate booking objects.
4. Accept the customer's travel mode and journey details.
5. Create the appropriate booking object based on the selected option.
6. Display the booking confirmation details.

Result: Successfully implemented the Factory Method concept to create Flight, Train, and Bus booking objects based on user selection.

4. Overall Conclusion

Both design patterns were successfully implemented using Python. The Singleton Pattern ensures a single shared instance, while the Factory Method approach simplifies object creation and improves code organization.

For your assignment document: Arrange the content as Aim → Objective → Procedure → Source Code → Output Screenshot → Result for each program.

1. Singleton - Banking Management System Code :

```
class BankManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.bank_name = "MyBank"
            cls._instance.balance = 10000
        return cls._instance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"₹{amount} deposited successfully.")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if amount <= 0:
```

```
            print("Invalid withdrawal amount.")
        elif amount > self.balance:
            print("Insufficient balance.")
        else:
            self.balance -= amount
            print(f"₹{amount} withdrawn successfully.")

    def check_balance(self):
        print(f"Current Balance: ₹{self.balance}")


# Creating two objects
bank1 = BankManager()
bank2 = BankManager()

print("===== SINGLETON BANKING SYSTEM =====")
print("Bank Name:", bank1.bank_name)

print("\nChecking Singleton Pattern:")
print("bank1 and bank2 are same instance:", bank1 is bank2)

while True:
    print("\n===== BANKING MENU =====")
    print("1. Deposit Money")
    print("2. Withdraw Money")
    print("3. Check Balance")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        try:
            amount = float(input("Enter deposit amount: ₹"))
            bank1.deposit(amount)
        except ValueError:
            print("Please enter a valid number.")

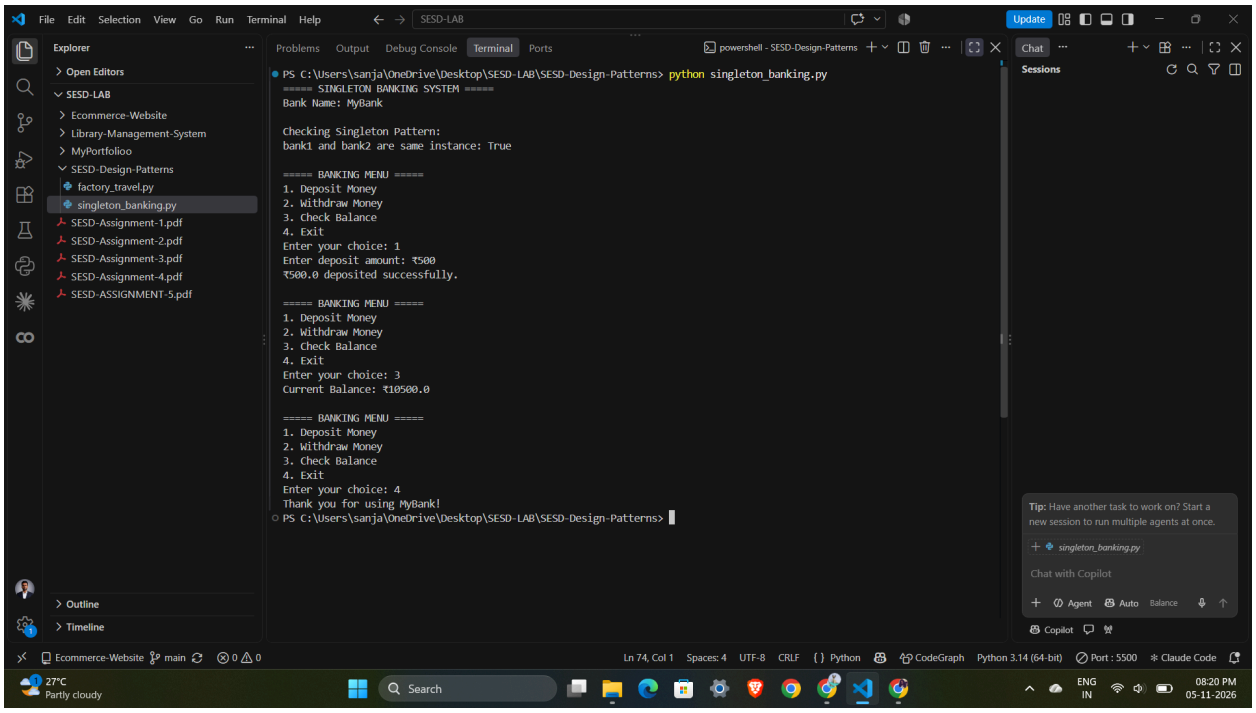
    elif choice == "2":
        try:
            amount = float(input("Enter withdrawal amount: ₹"))
            bank1.withdraw(amount)
        except ValueError:
            print("Please enter a valid number.")
```

```
    elif choice == "3":
        bank2.check_balance()

    elif choice == "4":
        print("Thank you for using MyBank!")
        break

    else:
        print("Invalid choice. Try again.")
```

Output :



---

2. Factory Method – Travel Booking System Code :

```
from abc import ABC, abstractmethod


# Abstract Booking Class
class Booking(ABC):

    @abstractmethod
    def book_ticket(self, name, source, destination):
```

```
        pass


# Flight Booking
class FlightBooking(Booking):

    def book_ticket(self, name, source, destination):
        print("\n===== FLIGHT TICKET =====")
        print("Passenger Name:", name)
        print("From:", source)
        print("To:", destination)
        print("Travel Mode: Flight")
        print("Booking Status: Confirmed")


# Train Booking
class TrainBooking(Booking):

    def book_ticket(self, name, source, destination):
        print("\n===== TRAIN TICKET =====")
        print("Passenger Name:", name)
        print("From:", source)
        print("To:", destination)
        print("Travel Mode: Train")
        print("Booking Status: Confirmed")


# Bus Booking
class BusBooking(Booking):

    def book_ticket(self, name, source, destination):
        print("\n===== BUS TICKET =====")
        print("Passenger Name:", name)
        print("From:", source)
        print("To:", destination)
        print("Travel Mode: Bus")
        print("Booking Status: Confirmed")


# Factory Method
class BookingFactory:

    @staticmethod
    def create_booking(choice):
```

```
        if choice == "1":
            return FlightBooking()

        elif choice == "2":
            return TrainBooking()

        elif choice == "3":
            return BusBooking()

        else:
            return None


# Main Program
while True:

    print("\n===== TRAVEL BOOKING SYSTEM =====")
    print("1. Flight Booking")
    print("2. Train Booking")
    print("3. Bus Booking")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "4":
        print("Thank you for using Travel Booking System!")
        break

    booking = BookingFactory.create_booking(choice)

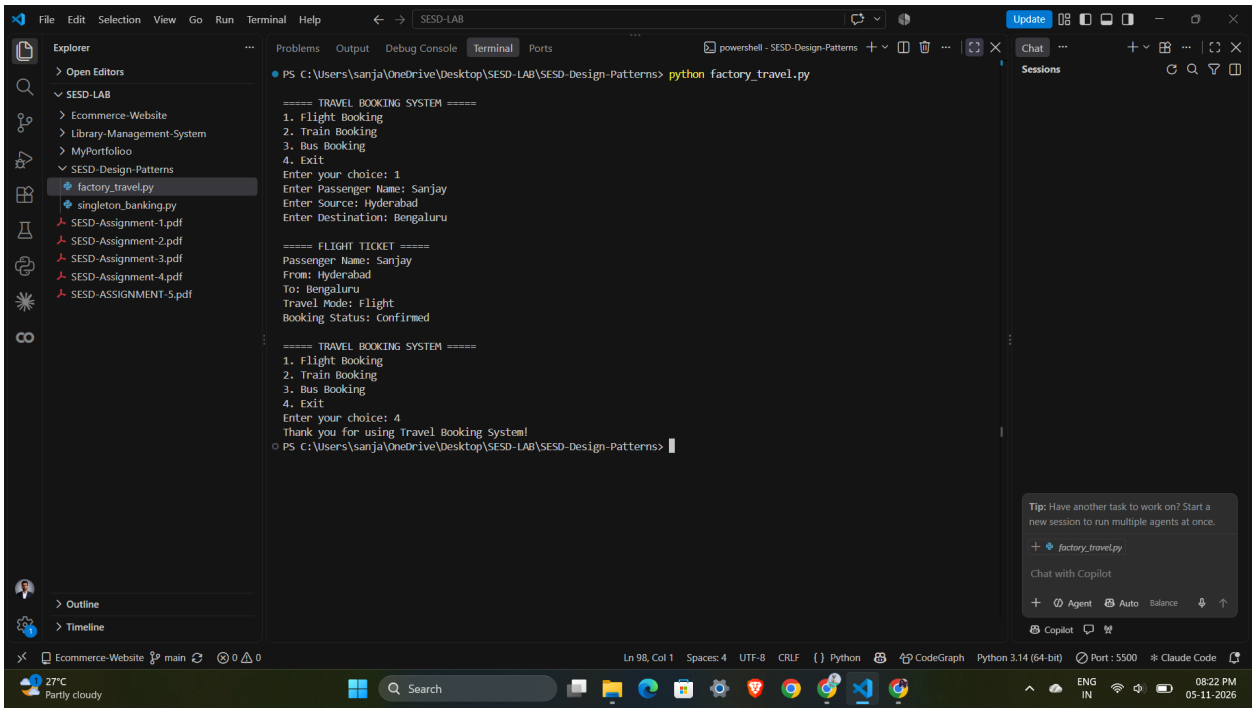
    if booking is None:
        print("Invalid choice! Please try again.")
        continue

    name = input("Enter Passenger Name: ")
    source = input("Enter Source: ")
    destination = input("Enter Destination: ")

    if not name.strip() or not source.strip() or not destination.strip():
        print("All details are required. Please try again.")
        continue

    booking.book_ticket(name, source, destination)
```

Output :


File Edit Selection View Go Run Terminal Help
SESD-LAB
Update
Explorer
Open Editors
SESD-LAB
Ecommerce-Website
Library-Management-System
MyPortfolioo
SESD-Design-Patterns
factory_travel.py
singleton_banking.py
SESD-Assignment-1.pdf
SESD-Assignment-2.pdf
SESD-Assignment-3.pdf
SESD-Assignment-4.pdf
SESD-ASSIGNMENT-5.pdf
Problems Output Debug Console Terminal Ports
powershell - SESD-Design-Patterns
PS C:\Users\sanja\OneDrive\Desktop\SESD-LAB\SESD-Design-Patterns> python factory_travel.py
===== TRAVEL BOOKING SYSTEM =====
1. Flight Booking
2. Train Booking
3. Bus Booking
4. Exit
Enter your choice: 1
Enter Passenger Name: Sanjay
Enter Source: Hyderabad
Enter Destination: Bengaluru
===== FLIGHT TICKET =====
Passenger Name: Sanjay
From: Hyderabad
To: Bengaluru
Travel Mode: Flight
Booking Status: Confirmed
===== TRAVEL BOOKING SYSTEM =====
1. Flight Booking
2. Train Booking
3. Bus Booking
4. Exit
Enter your choice: 4
Thank you for using Travel Booking System!
PS C:\Users\sanja\OneDrive\Desktop\SESD-LAB\SESD-Design-Patterns>
Chat
Sessions
Tip: Have another task to work on? Start a new session to run multiple agents at once.
factory_travel.py
Chat with Copilot
Agent
Auto
Balance
Copilot
Outline
Timeline
Ecommerce-Website
main
Ln 98, Col 1
Spaces: 4
UTF-8
CRLF
Python
CodeGraph
Python 3.14 (64-bit)
Port : 5500
Claude Code
27°C
Partly cloudy
Search
ENG
IN
08:22 PM
05-11-2026